Dynamic Structured Grammatical Evolution for House Price Prediction

John Foley

Project Overview

Our project goal is to use grammatical evolution, or more specifically an enhanced variation called dynamic structured GE to predict house prices. We use a Washington House Price dataset which consists of 4.6k samples. The goal is to evolve an expression tree that combines input features as well as mathematical functions to create a model that accurately predicts house prices.

Literature Review

Related Work

Previous studies on house price prediction using evolutionary methods have taken different approaches. In a paper by Azimlu et al., they used clustering (K-means, DBSCAN) before applying genetic programming to each cluster separately, achieving good results especially on smaller datasets [4]. In another paper by Shekarian et al., they applied GEP (Gene Expression Programming) with linear chromosomes that decode into expression trees, reporting R^2 of 0.92 compared to traditional regression methods [3]. Our DSGE implementation differs in two main ways: instead of clustering the data or using standard GP tree representations, we use Dynamic Structured GE which maps genotypes to phenotypes with no redundancy through per-nonterminal gene arrays [1]. We also apply a single evolved model to the whole dataset, relying on caching and structural constraints rather than ensemble or clustering approaches to handle the complexity.

Grammar Guidance

In our initial search for a grammar, we were still focusing on standard grammatical evolution. To the best of our knowledge, we could not find papers detailing standard grammatical evolution for

house prices, presumably as GP naturally presents itself as a well-suited, easier solution. The one paper which benchmarked GE on house prices used an enhanced version - Dynamic Structured GE. Our initial grammar was identical to that benchmarked, with the only exception being our variables are features from our Washington house dataset [2], while they base their variables off features from Boston House prices [1]. It may be noted that other than these variables, the grammar is identical to the grammar used to solve Quartic Symbolic Regression [1]. We would later decide to add constants to our grammar, which is discussed later.

Dynamic Structured Grammatical Evolution

Two well established issues with GE are its low locality and redundancy in genotypes [1].

- **Low locality** - a small modification to a genotype can completely change the corresponding phenotype.
- **Redundancy** - multiple genotypes mapping to the same phenotype, resulting in duplicate evaluations.

An early approach to resolve such issues was structured GE. Structured GE fundamentally changes how a GE is stored to improve locality and eliminate redundancy. However in doing so, it removes recursion which limits the possible solution, particularly for a grammar as small as ours. Dynamic structured GE can allow recursion as it builds each gene array on the fly (dynamically). Dynamic structured GE is core to our implemented solution, and resulted in drastically improved results as it eliminated redundancy.

System Design / Architecture

<u>Programming Language</u>
The programming language of choice for our project is Python and this decision was based on both of us being familiar with the language. We wanted to focus on the complexity of the algorithm at hand, not another language. Python also has very good library support. We recognise the language's speed as an issue, however we decided to focus on optimization later through multiprocessing and caching.

<u>Code Organisation</u>
Our code follows a functional programming paradigm. The diagram below shows a high level pipeline for how the algorithm runs. The evaluation code overview shows the function used and control flow of the multithreaded evaluation process.

Within the codebase main.py is the entry point that loads the dataset, calls the GE loop and then evaluates and visualizes the best individuals. Ge_main.py orchestrates the generations, by initialising the population, evaluating it, creating new populations with elitism and offspring, and returning the best individuals after x generations. Evaluation.py includes all functions required

for evaluation, population.py includes functions for initialising the population and mapping, genetic_operators.py includes functions for mutating and crossover and models.py contains structured classes for our phenotype trees, config and grammar.

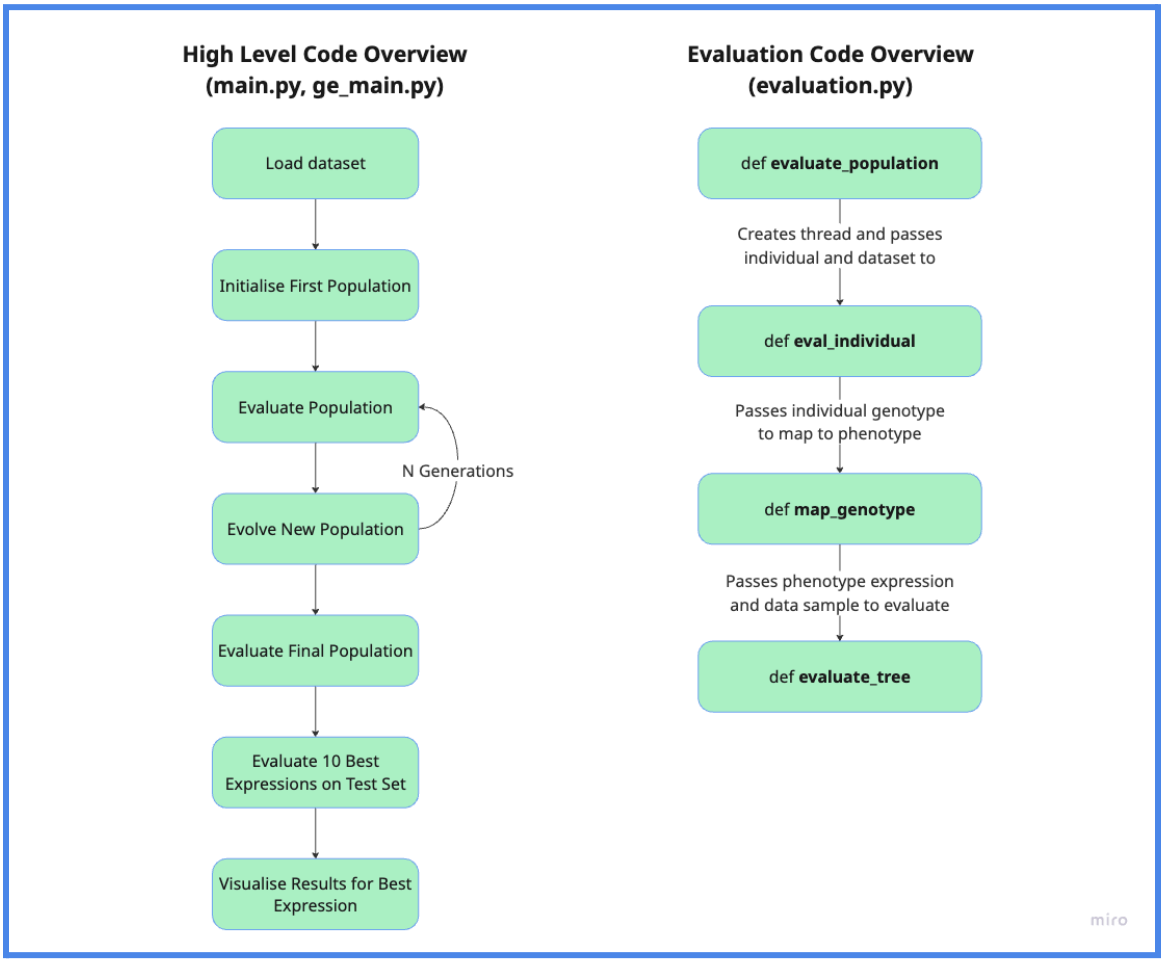

Figure 1: High Level Code Overview

Evaluating On Entire Train Set

When evaluating an individual's fitness, we evaluate the entire test set every time. This includes 3600 samples and hence is an expensive operation. An issue we were having when evaluating on a smaller test set was that the same expression could produce varying results when using smaller sample sizes. This is likely due to the variance of values in our dataset. By doing this we ensure not only accurate but also deterministic fitness values which is important for us when caching fitness values.

Implementation

The section to follow will give a high level overview of how we initialise and apply genetic operators in accordance with Dynamic Structured GE.

def initialise_individual

Initialising an individual in DSGE is a much more structured process than in standard GE, where bitstrings are initialised randomly. Instead a genotype is split into genes, one for each non-terminal. Each gene holds an array which represents indexes of the production rule to choose. We recursively build out the non-terminals until we are left with terminals.

```
def initialise_individual(grammar, start_nt, max_depth, rng=random):
    """
    Create a structured genotype: dict mapping non-terminals to lists of chosen productions.
    This aligns with DSGE where each non-terminal has its own gene list.
    """
    genotype = {nt: [] for nt in grammar.keys()}
    def expand(nt, depth):
        idx = choose_production(grammar, nt, depth, max_depth)
        genotype[nt].append(idx) # append index of rule chosen
        prod = grammar[nt][idx]
        for sym in prod:
            if sym in grammar: # recursively built out non-terminals
                expand(sym, depth+1)
    expand(start_nt, 0)
    return genotype # genotype is dict of lists of production indices for each non-terminal
```

Figure 2: Initialise Individual Snippet

def crossover_genotypes

Crossover in DSGE differs from standard GE due to its different genotype representation. Crossover is applied at a gene level in DSGE. The code below shows we give a 50/50 chance of a gene being swapped between two parents.

```
for gene in keys:
    take_from_p2 = rng.random() < 0.5  # mask bit for this gene
    g1 = parent1.get(gene, [])
    g2 = parent2.get(gene, [])

    if take_from_p2:
        # child1 takes P2's whole gene, child2 takes P1's
        child1[gene] = list(g2)
        child2[gene] = list(g1)
    else:
        # child1 takes P1's whole gene, child2 takes P2's
        child1[gene] = list(g1)
        child2[gene] = list(g2)
return child1, child2
```

Figure 3: Gene Crossover Snippet

def mutate_genotype

The mutation approach proposed in the original DSGE paper suggests mutating a single production for a single gene [1]. Our initial implementation made one mutation per genotype, but

the code was later refactored to allow more. This would allow us to later test a larger number of mutations and evaluate its effect. One noticeable change when making more mutations is we see the best genotype per population changes more frequently.

```python
def mutate_genotype(genotype, max_depth, mutations=1, rng=random):
    new_genotype = {nt: list(genes) for nt, genes in genotype.items()} # deep copy

    # Build list of all possible (non-terminal, position) candidates
    all_candidates = [(nt, pos) for nt, genes in new_genotype.items() for pos in range(len(genes))]
    num = min(mutations, len(all_candidates)) # incase user configs mutations > available genes
    chosen = rng.sample(all_candidates, num) # get gene and position pairs to mutate

    for nt, pos in chosen:
        genes = new_genotype[nt] # get gene list for non-terminal
        max_choices = len(GRAMMAR[nt]) # number of possible productions

        old_idx = genes[pos] # current production index
        new_idx = choose_production(GRAMMAR, nt, 0, max_depth) # new prod index

        if new_idx == old_idx and max_choices > 1: # ensure a different rule choice
            new_idx = (old_idx + 1) % max_choices
        genes[pos] = new_idx
    return new_genotype
```

Figure 4: Mutate Genotype Snippet

Pre-Operators

As mentioned in our grammar guidance section, we base our grammar off of that proposed in [1]. Included in this grammar are the following pre-operators: sin, cos, exponent, log, inverse. Within these operators, certain guardrails had to be implemented to avoid exploding values and dividing by zero issues. For example, exponent (e power x), uses the max of x and 70 to avoid exploding values. A clamp function limits values at a max value also. A safe log function is implemented by taking the max of x and a tiny epsilon value, as x must be greater than zero. Safe inverse is implemented by avoiding a divide by zero, while preserving sign integrity.

```python
EPS = 1e-10
MAX_MAG = 1e6

def clamp(v, m=MAX_MAG): # max magnitude in either direction
    return max(min(v, m), -m)

PRE_OPS = {
    'sin': lambda x: math.sin(x),
    'cos': lambda x: math.cos(x),
    'exp': lambda x: clamp(math.exp(min(x, 70))), # min x, 70 to avoid overflow
    'log': lambda x: math.log(max(x, EPS)), # log undefined for <=0, use EPS
    'inv': lambda x: clamp(1.0 / max(abs(x), EPS) * (1 if x >= 0 else -1)) # safe
}
```

Figure 5: Safe Pre-Operators

Note the operator set consists of addition, subtraction, multiplication and a safe division.

Constant Set
The initial constant set used was that suggested when benchmarking DSGE for the Boston House Problem [1]. That is the only values in the var set were the features from the dataset as well as the integer value 1. Note that within our dataset, we used fourteen features, three of which are one hot encoded values for strings.

Later in the implementation we add numeric values to our constant set. All prime numbers from 1-50 were added, the rationale behind this being the algorithm can combine other numbers to produce non-prime values. The results of adding these values will be discussed later.

Innovation / Features

Dynamic Structured Grammatical Evolution
A key innovation of our implementation was DSGE. Initially we had not planned to use DSGE, but as we analysed and evaluated our initial implementation of standard GE, we noticed redundancy was seriously affecting our population. Our pre DSGE implementation can be viewed on this following branch. After looking through research papers, we came across DSGE as a perfect solution to eliminate our redundancy issues while improving locality. This is the main innovation added to our implementation based on relevant literature.

Multithreading
As mentioned earlier, issues with Python include its performance and we would look to optimize our implementation later. When looking for bottlenecks in our generation loop, it became evident that evaluation took up over 95% of the time. Before multithreading, evaluation on a population took roughly 18 seconds for a max tree depth of 5 and population size 1000. After introducing a process pool, the evaluation time reduced over 80% for a time of under 3 seconds. Note that reduction time will be dependent on hardware and processes available and that there were 10 processes available for the test detailed above.

```
def evaluate_population(population, X, y, cfg):
    start = time.perf_counter()
    with Pool(processes=cpu_count()) as pool:
        # Create argument tuples for each individual
        args_list = [(ind, X, y, cfg) for ind in population]
        results = pool.map(_eval_individual_wrapper, args_list)
    logger.info("Evaluation time: %.4fs", time.perf_counter() - start)
    return results
```

Figure 5: Multithreaded Process Pool For Evaluation

Unique Individuals In Population
Although DSGE ensures different genotypes cannot map to the same phenotype, an existing issue is that we could still create two of the same genotypes within a population. Having

identical genotypes in a population wastes precious space within our population. We also noticed that our approach of creating the new population based on the top 7% could result in more duplicates. Therefore when generating a new population, we temporarily add all genomes to a set. While adding to the population, we ensure the new genotype does not exist. If it does, we simply mutate the genotype until, we arrive at a unique genotype for that population, or we have reached the 100th attempt at finding a unique genotype. In the case we use up the 100 attempts and still have not found a unique genotype, we add the 100th attempt to the population and move on to the next genotype. Putting a cap on the number of retries keeps the algorithm from getting stuck in an endless search for new genotypes, which prevents later generations from effectively hanging while still discouraging excessive duplicates

```
# Create set to track unique genomes in new population
genome_set = {genome_to_tuple(ind['genotype']) for ind in new_pop}

# --- Reproduction ---
while len(new_pop) < cfg.population_size:
    p1, p2 = random.choice(parents), random.choice(parents)

    c1, c2 = crossover_individuals(p1, p2)
    c1g = mutate_genotype(c1['genotype'], max_depth=cfg.max_depth)
    c2g = mutate_genotype(c2['genotype'], max_depth=cfg.max_depth)

     # Try to find unique genome for c1, with max retries to prevent infinite loop
    max_retries = 100
    retries = 0
    while genome_to_tuple(c1g) in genome_set and retries < max_retries:
        c1g = mutate_genotype(c1g, max_depth=cfg.max_depth)
        retries += 1

    genome_set.add(genome_to_tuple(c1g))
    new_pop.append({'genotype': c1g, 'phenotype': None, 'fitness': None})

    if len(new_pop) < cfg.population_size:
        # Ensure c2g is unique - keep mutating until it is
        retries = 0
        while genome_to_tuple(c2g) in genome_set and retries < max_retries:
            c2g = mutate_genotype(c2g, max_depth=cfg.max_depth)
            retries += 1
        genome_set.add(genome_to_tuple(c2g))
        new_pop.append({'genotype': c2g, 'phenotype': None, 'fitness': None})
```

Figure 6: Ensure Unique Individuals In Population

Caching

Another key optimization strategy is our caching mechanisms. The first of these is a genotype to phenotype cache, which maps a genotype to a phenotype expression. This cache was introduced to avoid remappings of genotypes to phenotypes. Note that although we ensure no duplicate genotypes in a single population (discussed above), genotypes can still duplicate from generation to generation and our caching avoids remapping in such cases.

The second cache we have introduced is a phenotype (expression) to fitness cache. This cache provides large performance boosts, as evaluating an expression on all 3600 train samples is an expensive operation. As with the genotype to phenotype cache, the phenotype to fitness cache is used for phenotypes repeated between generations. It is worth noting that technically, a single genotype to fitness cache would suffice with our newly added implementation of DSGE, however we have opted to stay with two caches as verifying both caches are the same size ensures we have one to one mappings between genotypes and phenotypes and the correct implementation of DSGE.

Test Suite

A comprehensive test suite has been added to ensure implementation correctness. The test suite includes 28 unit tests, giving us confidence in implementation correctness. Details on how to run the tests can be found in our ReadME.md .

Results / Performance / Analysis

Computational Results

A feature of our implementation was its evolution from single threaded to multithreaded and then to multiprocessed. Multiprocessing gave us major performance boosts, from roughly 18 seconds to less than 3 seconds for a population size 1000 and tree size of 5. This demonstrates an 85% speed increase for evaluating a population. The change commit from single-threaded to multi-threaded can be seen here, while the commit changing from multi-threaded to multi-processed can be found here.

```
INFO: Evaluation time: 3.1805s
INFO: Gen 0: Best Fitness 301457.7973 Expr:
INFO: Evaluation time: 2.0679s
INFO: Gen 1: Best Fitness 288712.8610 Expr:
INFO: Evaluation time: 2.6132s
INFO: Gen 2: Best Fitness 193410.6903 Expr:
rooms) - (19.0 + floors))))
INFO: Evaluation time: 2.7449s
INFO: Gen 3: Best Fitness 193410.6903 Expr:
rooms) - (19.0 + floors))))
INFO: Evaluation time: 2.8517s
```

Figure 7: Post Multi-Process Time Improvements

Redundancy Improvement Results

As mentioned previously a core aspect of our innovation was to remove redundancy. This was done through structured GE. Our way of validating this is ensuring our genotype to expression and phenotype to fitness cache sizes are the same. Every new genotype and phenotype encountered is stored in the cache, hence verifying both cache sizes equal verifies our 1:1 mapping and elimination of redundancy. The results below show redundancy issues pre DSGE and the 1:1 mapping post DSGE.

```
Gen 9: Best Fitness 552628.13 Expr: 11 * sqft_living * 23 + 16 * city_num * 16
Generation 9 completed in 6.6742s
Expr cache size: 8200, Fitness cache size: 3424
```

Figure 8: Redundancy Pre DSGE

```
INFO: Fitness cache size: 10564
INFO: Genome to expression cache size: 10564
```

Figure 9: Redundancy Eliminated Post DSGE

Constant Set Additions

As rationalised previously, we decided to test performance with the addition of prime numbers under fifty. We found that by introducing these prime numbers to the set allowed expression to approximate better in early generations. We ran 20 generations for a population size of 1000 and max tree depth of 5. The figures below show the initial best genotype has a RMSE of 352266 with the primes and 425325 without the primes.

```
INFO: Evaluation time: 3.2157s
INFO: Gen 0: Best Fitness 425325.7889 Expr: exp((((sqft_living - yr_built) - bathrooms) - 1.0))
```

Figure 10: Initial Generation Without Primes

```
INFO: Evaluation time: 3.4997s
INFO: Gen 0: Best Fitness 352266.6261 Expr: inv((bedrooms / exp((19.0 * bedrooms))))
```

Figure 11: Initial Generation With Primes

After 20 generations, we saw significant improvements in the prediction and loss values. There is a RMSE difference of over 30,000 and on average the price prediction is $35,000 closer. Note the full logs for these runs can be found in the logs folder of our repository.

```
INFO: TEST SET EVALUATION SUMMARY
INFO: ========================================
INFO:   Test samples: 887
INFO:   Best individual on test set: Rank 4
INFO:   Test RMSE: 223667.9437
INFO:   Average Absolute Error: $180256.36
```

Figure 12: Final Result Without Prime Numbers

```
INFO: TEST SET EVALUATION SUMMARY
INFO: ========================================
INFO:   Test samples: 887
INFO:   Best individual on test set: Rank 1
INFO:   Test RMSE: 192003.4908
INFO:   Average Absolute Error: $143440.43
```

Figure 13: Final Result With Prime Number

Another metric we were interested in was how often the best individual changes - this gives us an idea of how well the grammar is learning. No significant differences were recorded. Without the prime numbers it changes 11 times out of 20 generations, while with the prime numbers the best individual changed 12 times out of 20 generations.

Final Results

Our final results came from a run with 100 generations, a population size of 1000, max tree depth of 5 and using prime numbers. Note a full log for this run can be found in the log folder also. The graph below shows how function is beginning to evolve and fit the points.

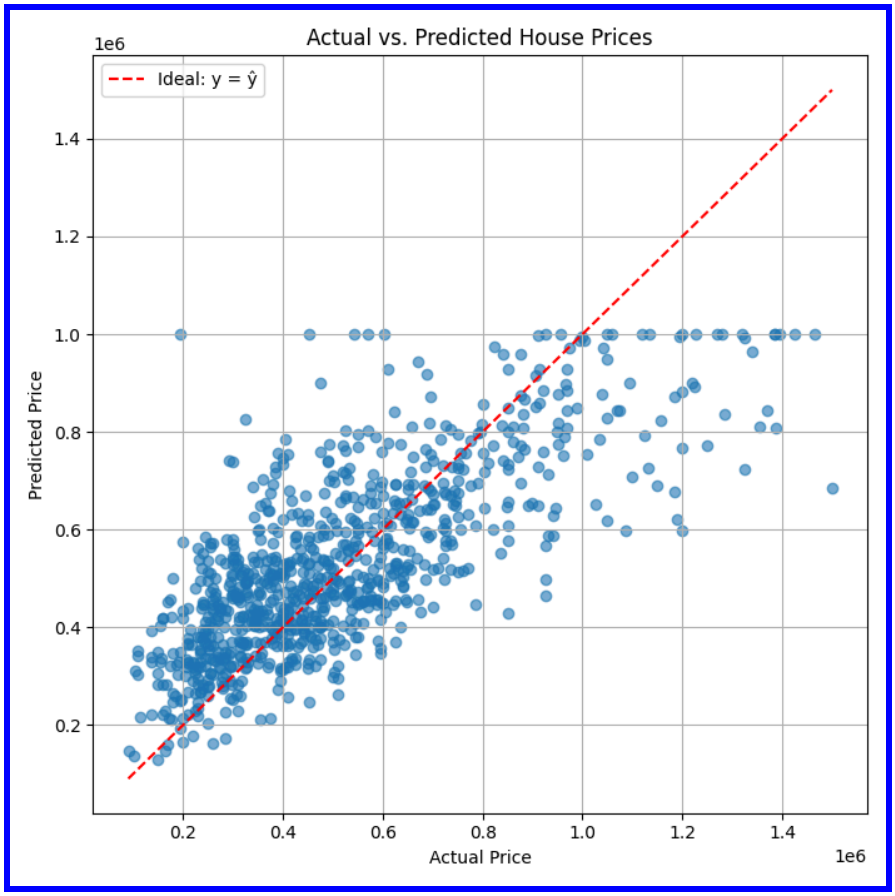

Figure 14: Actual vs Predicted Price

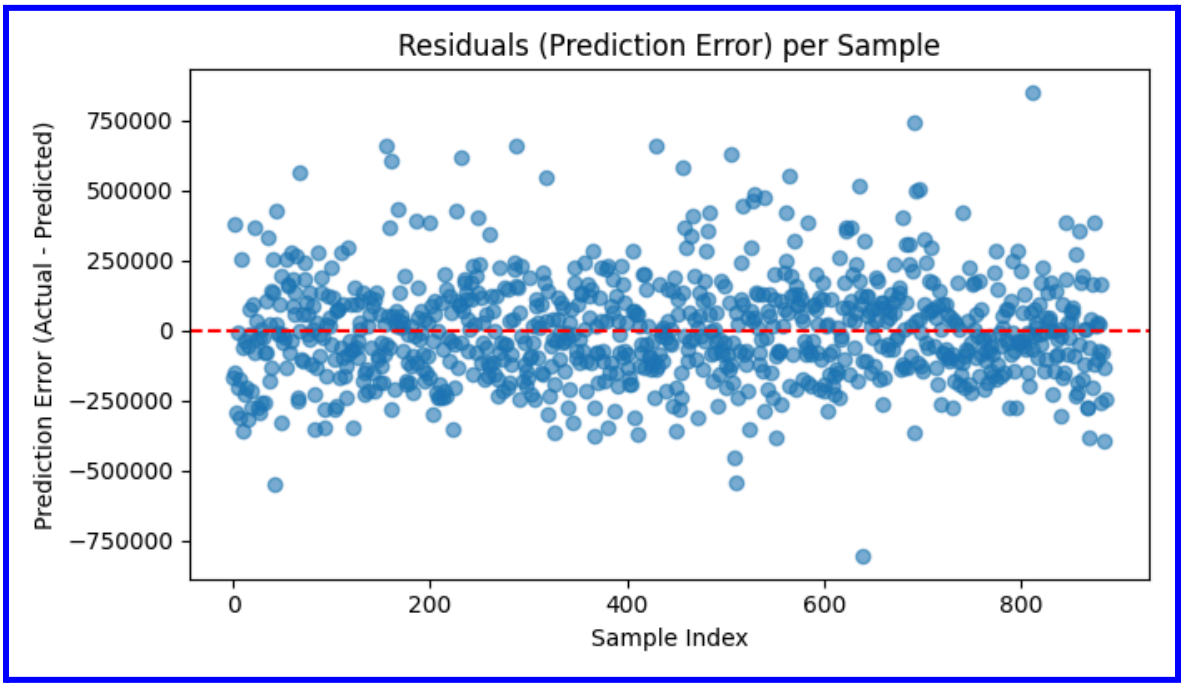

Figure 15: Error - Actual Minus Predicted

We can see in figure [ref] above, that our line fits our data, and that our function is fitting to the middle of the datapoints. The majority of our data points are predicted to be within $100,000, which as seen from the figure above. When evaluating on our test set, we pick the best 10 performing genotypes from the train and evaluate them on the test. In figure [ref] below, we see a RMSE of 171,000 and an average prediction error of $130,000.

```
INFO: TEST SET EVALUATION SUMMARY
INFO: ========================================
INFO:   Test samples: 887
INFO:   Best individual on test set: Rank 2
INFO:   Test RMSE: 171762.9636
INFO:   Average Absolute Error: $130099.64
```

Figure 16: Prediction Metrics

References

1. Lourenço, Nuno, et al. “Structured Grammatical Evolution: A Dynamic Approach.” *Handbook of Grammatical Evolution*, edited by Conor Ryan et al., Springer International Publishing, 2018, pp. 137–61. *Springer Link*, https://doi.org/10.1007/978-3-319-78717-6_6.
2. *King Country Housing Dataset* https://www.kaggle.com/datasets/vallabhadattap/kingcountyhousing. Accessed 12 Dec. 2025.
3. Shekarian, Ehsan, and Alireza Fallahpour. “Predicting House Price via Gene Expression Programming.” *International Journal of Housing Markets and Analysis*, vol. 6, no. 3, July 2013, pp. 250–68. *DOI.org (Crossref)*, https://doi.org/10.1108/IJHMA-08-2012-0039.
4. Azimlu, Fateme, et al. “House Price Prediction Using Clustering and Genetic Programming along with Conducting a Comparative Study.” *Proceedings of the Genetic and Evolutionary Computation Conference Companion* [New York, NY, USA], GECCO ’21, 2021, pp. 1809–16, https://doi.org/10.1145/3449726.3463141.